

Task 3: Dataset Preparation and Preprocessing for Deep Learning

Dataset: CIFAR-10

Objective

Prepare and preprocess the CIFAR-10 image dataset for deep learning by performing exploratory data analysis, data-quality checks, normalization, augmentation, visualization, and train-validation-test splitting.

Tools and Libraries

- Python
- TensorFlow / Keras
- NumPy
- Pandas
- Matplotlib
- Seaborn

Dataset

CIFAR-10 contains 60,000 color images of size 32×32 pixels belonging to 10 classes.

1. Import Libraries

Source Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow.keras.datasets import cifar10
from tensorflow.keras import layers

print("TensorFlow version:", tf.__version__)
np.random.seed(42)
tf.random.set_seed(42)
```

Output

```
TensorFlow version: 2.20.0
```

2. Load the CIFAR-10 Dataset

Source Code

```
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

print("Original training images:", x_train_full.shape)
print("Original training labels:", y_train_full.shape)
```

```
print("Test images:", x_test.shape)
print("Test labels:", y_test.shape)
```

Output

```
Original training images: (50000, 32, 32, 3)
Original training labels: (50000, 1)
Test images: (10000, 32, 32, 3)
Test labels: (10000, 1)
```

3. Dataset Overview

Source Code

```
print("Total images:", len(x_train_full) + len(x_test))
print("Image shape:", x_train_full.shape[1:])
print("Number of classes:", len(class_names))
print("Data type:", x_train_full.dtype)
print("Minimum pixel value:", x_train_full.min())
print("Maximum pixel value:", x_train_full.max())
```

Output

```
Total images: 60000
Image shape: (32, 32, 3)
Number of classes: 10
Data type: uint8
Minimum pixel value: 0
Maximum pixel value: 255
```

4. Exploratory Data Analysis (EDA)

Source Code — Sample images from each class

```
plt.figure(figsize=(15, 6))
for class_id in range(10):
    index = np.where(y_train_full.flatten() == class_id)[0][0]
    plt.subplot(2, 5, class_id + 1)
    plt.imshow(x_train_full[index])
    plt.title(class_names[class_id])
    plt.axis("off")
plt.suptitle("One Sample Image from Each CIFAR-10 Class", fontsize=16)
plt.tight_layout()
plt.show()
```

Screenshot / Output

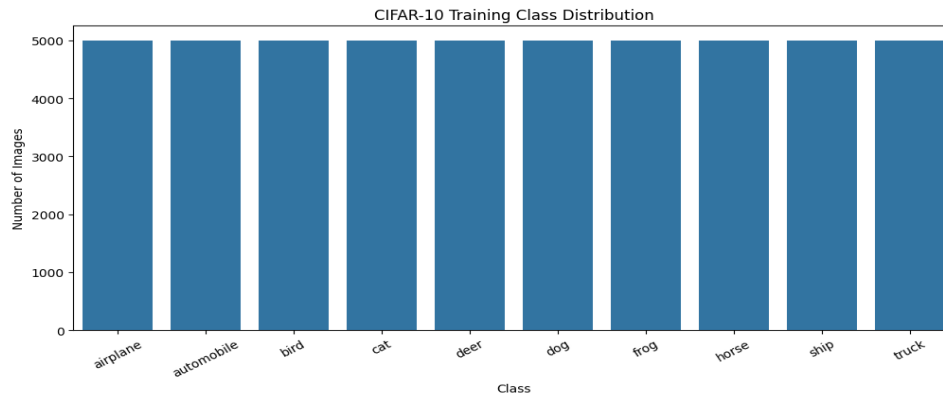


Source Code — Class distribution

```
class_counts = pd.Series(y_train_full.flatten()).value_counts().sort_index()
distribution_df = pd.DataFrame({"Class": class_names, "Number of Images": class_counts.values})
display(distribution_df)

sns.barplot(data=distribution_df, x="Class", y="Number of Images")
plt.title("CIFAR-10 Training Class Distribution")
plt.show()
```

Screenshot / Output

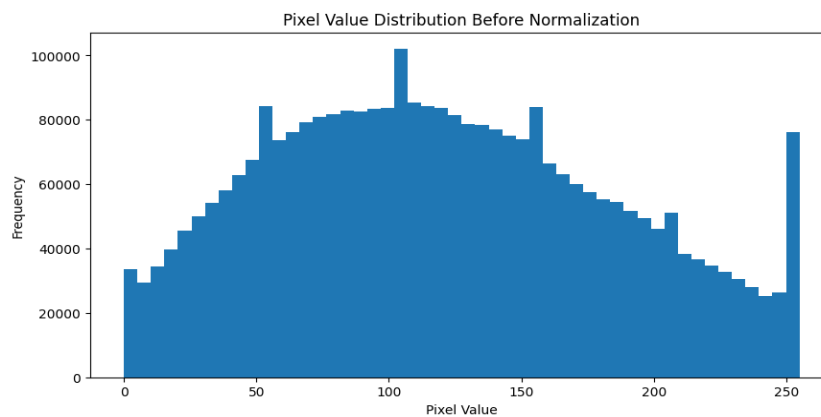


Each of the 10 classes contains exactly 5,000 images in the training set, confirming the dataset is perfectly balanced.

Source Code — Pixel value distribution

```
sample_pixels = x_train_full[:1000].reshape(-1)
plt.hist(sample_pixels, bins=50)
plt.title("Pixel Value Distribution Before Normalization")
plt.show()
```

Screenshot / Output



5. Data Quality Checks

The CIFAR-10 arrays were checked for missing values, invalid pixel values, invalid labels, unexpected image dimensions, and duplicate images within a manageable subset.

Source Code

```
print("Missing values in training images:", np.isnan(x_train_full).sum())
print("Missing values in test images:", np.isnan(x_test).sum())
print("Unique training labels:", np.unique(y_train_full))
print("Training pixel range:", x_train_full.min(), "to", x_train_full.max())
print("Training image shape valid:", x_train_full.shape[1:] == (32, 32, 3))
```

Output

```
Missing values in training images: 0
Missing values in test images: 0
Missing values in training labels: 0
Missing values in test labels: 0

Unique training labels: [0 1 2 3 4 5 6 7 8 9]
Unique test labels: [0 1 2 3 4 5 6 7 8 9]

Training pixel range: 0 to 255
Test pixel range: 0 to 255

Training image shape valid: True
Test image shape valid: True
```

Source Code — Duplicate check

```
subset = x_train_full[:10000]
flat_subset = subset.reshape(len(subset), -1)
unique_count = np.unique(flat_subset, axis=0).shape[0]
print("Images checked:", len(subset))
print("Unique images:", unique_count)
print("Duplicate images in checked subset:", len(subset) - unique_count)
```

Output

```
Images checked: 10000
Unique images: 10000
Duplicate images in checked subset: 0
```

6. Train-Validation-Test Split

Source Code

```
validation_size = 5000
x_val = x_train_full[-validation_size:]
y_val = y_train_full[-validation_size:]
x_train = x_train_full[:-validation_size]
y_train = y_train_full[:-validation_size]

print("Training set:", x_train.shape, y_train.shape)
print("Validation set:", x_val.shape, y_val.shape)
print("Test set:", x_test.shape, y_test.shape)
```

Output

```
Training set: (45000, 32, 32, 3) (45000, 1)
Validation set: (5000, 32, 32, 3) (5000, 1)
Test set: (10000, 32, 32, 3) (10000, 1)
```

7. Image Preprocessing — Normalization

Source Code

```
x_train_norm = x_train.astype("float32") / 255.0
x_val_norm = x_val.astype("float32") / 255.0
x_test_norm = x_test.astype("float32") / 255.0

print("Training pixel range after normalization:", x_train_norm.min(), "to", x_train_norm.max())
print("Data type:", x_train_norm.dtype)
```

Output

```
Training pixel range after normalization: 0.0 to 1.0
Data type: float32
```

Screenshot / Output — Normalized images



8. Image Augmentation

Augmentation is applied only to training images; validation and test images are kept unaugmented.

Source Code

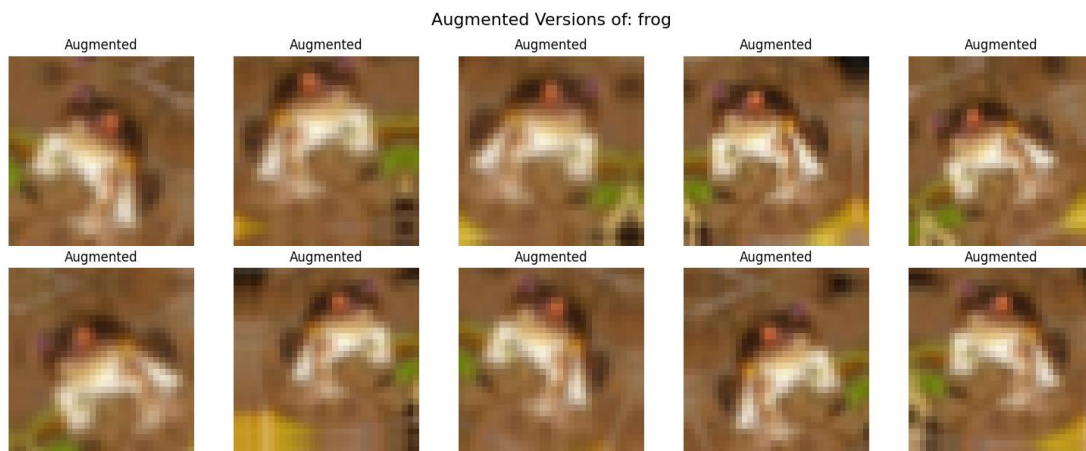
```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.RandomTranslation(0.1, 0.1)
], name="data_augmentation")

data_augmentation.summary()
```

Source Code — Visualize augmentations

```
sample_image = tf.expand_dims(x_train_norm[0], axis=0)
for i in range(10):
    augmented_image = data_augmentation(sample_image, training=True)[0]
    plt.subplot(2, 5, i + 1)
    plt.imshow(tf.clip_by_value(augmented_image, 0, 1))
    plt.axis("off")
plt.suptitle(f"Augmented Versions of: {class_names[int(y_train[0])}]")
plt.show()
```

Screenshot / Output



9. Prepare TensorFlow Datasets

Source Code

```
BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE

train_ds = tf.data.Dataset.from_tensor_slices((x_train_norm, y_train))
val_ds = tf.data.Dataset.from_tensor_slices((x_val_norm, y_val))
test_ds = tf.data.Dataset.from_tensor_slices((x_test_norm, y_test))

train_ds = train_ds.shuffle(10000, seed=42).batch(BATCH_SIZE).prefetch(AUTOTUNE)
val_ds = val_ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)
test_ds = test_ds.batch(BATCH_SIZE).prefetch(AUTOTUNE)

print("TensorFlow datasets created successfully.")
```

Output

```
TensorFlow datasets created successfully.
```

10. Verify the Preprocessed Dataset

Source Code

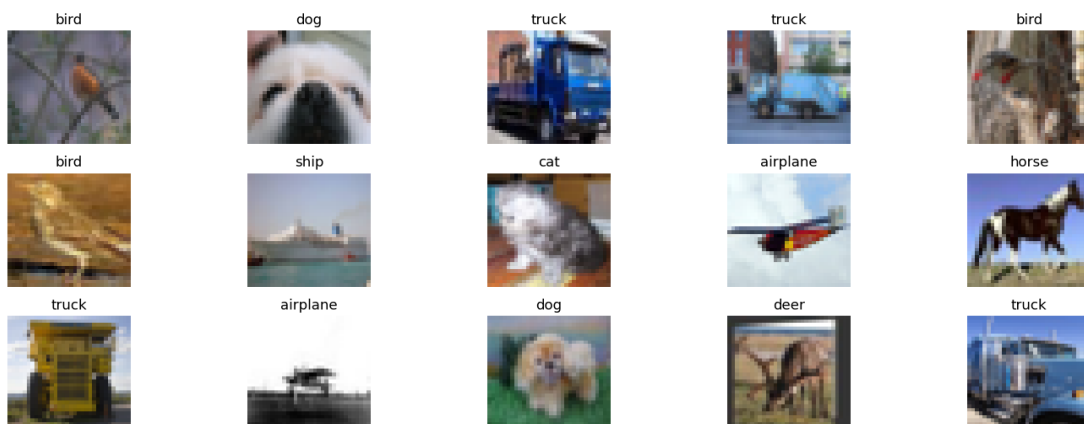
```
images_batch, labels_batch = next(iter(train_ds))
print("Batch image shape:", images_batch.shape)
print("Batch label shape:", labels_batch.shape)
print("Batch pixel range:", tf.reduce_min(images_batch).numpy(), "to",
      tf.reduce_max(images_batch).numpy())
print("Batch image dtype:", images_batch.dtype)
```

Output

```
Batch image shape: (64, 32, 32, 3)
Batch label shape: (64, 1)
Batch pixel range: 0.0 to 1.0
Batch image dtype: <dtype: 'float32'>
```

Screenshot / Output — Final preprocessed batch

Sample Images from Preprocessed Training Dataset



11. Preprocessing Summary

Step	Operation
Dataset loading	CIFAR-10 loaded using Keras
EDA	Image samples, dimensions, class distribution and pixel distribution
Data quality	Missing values, labels, dimensions, pixel ranges and duplicates checked
Train split	45,000 images
Validation split	5,000 images
Test split	10,000 images
Normalization	Pixel values scaled from 0–255 to 0–1
Augmentation	Horizontal flip, rotation, zoom and translation
TensorFlow pipeline	Batching, shuffling and prefetching

Important Preprocessing Decision

Data augmentation is applied only to the training data. Validation and test data are kept unaugmented so that evaluation represents the model's performance on the original data distribution.

12. Observations

1. CIFAR-10 contains RGB images with dimensions $32 \times 32 \times 3$ and ten target classes.
2. The dataset is approximately balanced across the ten classes.
3. Pixel values initially range from 0 to 255.
4. No missing values were found in the loaded CIFAR-10 arrays.
5. Labels are valid integer class IDs from 0 to 9.
6. Normalization scales pixel values to the range 0–1, making them more suitable for neural-network training.
7. Image augmentation creates additional variation in training samples and can help reduce overfitting.
8. The final dataset is divided into training, validation and test sets.
9. The resulting TensorFlow datasets are batched and prefetched for efficient deep-learning training.

13. Conclusion

The CIFAR-10 dataset was successfully prepared for deep learning. Exploratory analysis was performed, data quality was checked, images were normalized, training images were augmented, and the dataset was divided into training, validation and test sets. The final tf.data pipelines are ready to be used as input to a CNN or other image-based deep-learning model.

Github Repo:

https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task%203:%20Dataset%20Preparation%20and%20Preprocessing%20for%20Deep%20Learning